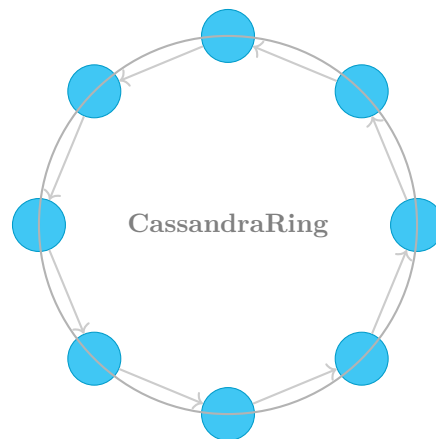


Wide-Column Stores & Apache Cassandra

Complete Exam-Ready Study Notes

F20BD / F21BD – Big Data Management

Heriot-Watt University



March 28, 2026

Contents

1	What Are Wide-Column Stores?	2
1.1	Where Do Wide-Column Stores Fit?	2
1.2	Row-Oriented vs. Column-Oriented Storage	2
2	The Wide-Column Data Model	4
2.1	Key Terminology (Hierarchy)	4
2.2	Column Family: Sparse and Flexible	4
2.3	Primary Key = Partition Key + Clustering Column	5
3	Apache Cassandra	6
3.1	History and Origins	6
3.2	Key Properties of Cassandra	6
3.3	Cassandra and the CAP Theorem	6
4	Cassandra Architecture	8
4.1	Consistent Hashing (How Data Is Distributed)	8
4.2	Write Path (How Cassandra Writes Data)	8
4.3	Compaction (Housekeeping)	9
4.4	Read Path (How Cassandra Reads Data)	9
4.5	Write Path: Consistency Levels	9
5	Bloom Filters	11
5.1	How Bloom Filters Work: Step by Step	11
5.1.1	Step 1: Setup	11
5.1.2	Step 2: Inserting an Element	11
5.1.3	Step 3: Checking if an Element Exists	12
6	Cassandra Query Language (CQL)	13
6.1	Keyspace Operations	13
6.2	Column Family (Table) Operations	13
6.3	INSERT, SELECT, UPDATE, DELETE	14
6.4	Querying Non-Primary-Key Columns	14
6.5	ALTER COLUMNFAMILY (Adding Columns)	15
7	Cassandra vs Relational Databases (RDBMS)	16
8	Wide Columns: Custom Types and Nested Data	17
8.1	Custom Types (User-Defined Types)	17
8.2	Using Maps for Multiple Addresses	18
9	DataStax Astra & Python Connection	19
9.1	Key Steps to Connect	19
10	Cassandra vs MongoDB	20
11	CQL Limitations	21
12	Quick Reference Cheat Sheet	22
12.1	CQL Commands At A Glance	22
12.2	Key Concepts Summary	22
12.3	Practice Questions	23

What Are Wide-Column Stores?

The Library Analogy Imagine a library where every bookshelf (= **column family** / **table**) stores books (= **rows**). In a *normal* library, every shelf has the exact same number of slots and the same labels (Author, Title, Year, Pages). But in a **wide-column library**, each book on the shelf can have *different* labels. One book might have “Author, Title, Year”, while the next book on the same shelf has “Author, Title, Publisher, ISBN” — and that is perfectly fine. No wasted empty slots.

A **wide-column store** is a type of NoSQL database that organises data into *rows* and *columns*, similar to a relational table — but with one crucial difference: **each row can have a different set of columns**. This makes it *sparse* (no wasted space for empty cells) and extremely flexible.

Formal Definition

A wide-column store is “**a sparse, distributed, persistent, multi-dimensional, sorted map.**”

Let’s break that down:

- **Sparse** — Not every row has a value for every column. No wasted storage.
- **Distributed** — Data is spread across many machines (nodes).
- **Persistent** — Data is saved to disk; it survives restarts.
- **Multi-dimensional** — Each cell can hold multiple pieces of information (name, value, timestamp).
- **Sorted Map** — Data is stored in sorted order by key, enabling fast lookups.

Where Do Wide-Column Stores Fit?

Wide-column stores are a sub-type of **NoSQL** databases. Here is the full landscape:

Category	Examples	Best For
Key-Value	Redis, Memcached	Simple lookups (caching, sessions)
Document	MongoDB, CouchBase	Rich, nested JSON-like data
Wide-Column	Cassandra, HBase	Massive-scale writes, time-series, IoT
Graph	Neo4J, Titan	Relationships (social networks)
RDF	Virtuoso, GraphDB	Linked data, semantic web

Row-Oriented vs. Column-Oriented Storage

How Data Is Physically Stored On Disk

Row-oriented (traditional RDBMS like MySQL): stores each *entire row* together on disk.

```
rowId1, firstName1, lastName1, phone1;
rowId2, firstName2, lastName2, phone2;
rowId3, firstName3, lastName3, phone3;
```

Wide-column (like Cassandra): groups all values for each *column* together on disk.

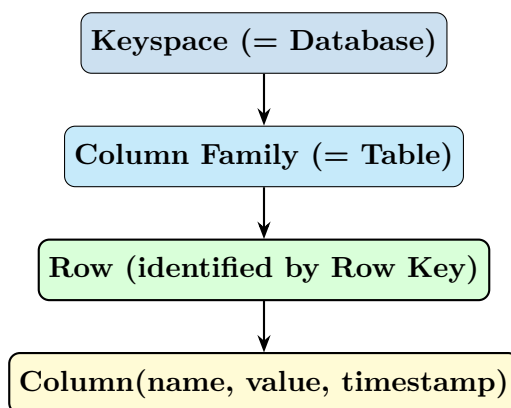
```
rowId1, rowId2, rowId3;
firstName1, firstName2, firstName3;
lastName1, lastName2, lastName3;
phone1, phone2, phone3;
```

Why does this matter? If you only need the `phone` column from a million rows, a column-oriented store reads *only* the phone data from disk. A row-oriented store must read every entire row, wasting time on `firstName`, `lastName`, etc. This makes wide-column stores **extremely fast for analytics and aggregations on specific columns**.

The Wide-Column Data Model

Key Terminology (Hierarchy)

Think of the data model as a nesting hierarchy — like folders inside folders:



The Four Levels — Know These for the Exam

1. **Keyspace** = The top-level container. Like a “database” in MySQL. Contains all your tables.
2. **Column Family** = A table. Contains rows. In CQL, you can use **TABLE** or **COLUMNFAMILY** interchangeably.
3. **Row** = One record. Identified by a unique **Row Key** (= Primary Key). Each row contains a set of columns.
4. **Column** = The smallest unit. A column has three parts: **name** (what it’s called), **value** (the data), and **timestamp** (when it was written).

Column = (name, value, timestamp) Every single cell in Cassandra stores three things: the column **name**, the **value**, and a **timestamp** showing when it was last written. The timestamp is used to resolve conflicts — the newest timestamp wins.

Column Family: Sparse and Flexible

The magic of wide-column stores is that **different rows can have different columns**. No NULLs needed — if a row doesn’t have a column, it simply doesn’t store it.

Performer Data

performer	born	country	died	founded	style	type	
John Lennon	1940	England	1980		Rock	artist	
Paul McCartney	1942	England			Rock	artist	
The Beatles		England		1957	Rock	band	

Notice: John Lennon has no **founded** value. Paul McCartney has no **died** or **founded**. The Beatles has no **born** or **died**. In an RDBMS, these would all be NULLs wasting space. In Cassandra, **those cells simply don’t exist**. Each row only stores the columns it actually uses.

Primary Key = Partition Key + Clustering Column

Primary Key Structure

In Cassandra, the **Primary Key** has two parts:

1. **Partition Key** — Determines *which node* in the cluster stores this row. Cassandra hashes the partition key to decide which machine the data goes to.
2. **Clustering Column(s)** (optional) — Determines the *sort order* of rows within the same partition. Data with the same partition key is stored together, sorted by the clustering column.

Hotel Analogy Think of the **partition key** as the *hotel building* you're assigned to — it decides which physical building (node) you go to. The **clustering column** is your *room number* — within that building, it determines exactly where your data sits (in sorted order).

Orders Table with Composite Primary Key

```
CREATE TABLE orders (
  customer_id UUID,      -- partition key
  order_date TIMESTAMP,  -- clustering column
  total DECIMAL,
  PRIMARY KEY (customer_id, order_date)
);
```

Here, `customer_id` is the partition key (all orders for one customer go to the same node). `order_date` is the clustering column (within that customer's data, orders are sorted by date automatically on disk).

Exam favourite: Partition Key vs Clustering Column

- **Partition Key:** Which node stores the data. Used for *data distribution*.
- **Clustering Column:** How data is sorted *within* a partition. Used for *ordering*.
- In PRIMARY KEY (A, B, C): A = partition key, B and C = clustering columns.
- In PRIMARY KEY ((A, B), C): (A, B) = composite partition key, C = clustering column.

Apache Cassandra

History and Origins

Cassandra was originally developed at **Facebook** by Avinash Lakshman (who also co-authored Amazon's Dynamo) and Prashant Malik. It was built to power Facebook's inbox search feature, handling billions of messages. Facebook open-sourced it in 2008, and it became a top-level Apache project in 2010.

The name "Cassandra" comes from Greek mythology — a Trojan prophet cursed so that nobody would believe her predictions.

Key Properties of Cassandra

The 8 Key Properties

1. **Free and open-source** — No licence fees.
2. **Distributed** — Data is spread across many machines.
3. **Wide Column Store** — Flexible column model (as described above).
4. **NoSQL** — Not a traditional relational database.
5. **Masterless replication** — Every node is equal; no single boss node.
6. **Low latency** — Very fast reads and writes.
7. **Elastic** — Add or remove nodes with zero downtime.
8. **Tuneable consistency** — You choose how consistent vs. how available you want to be.

What Makes Cassandra Special?

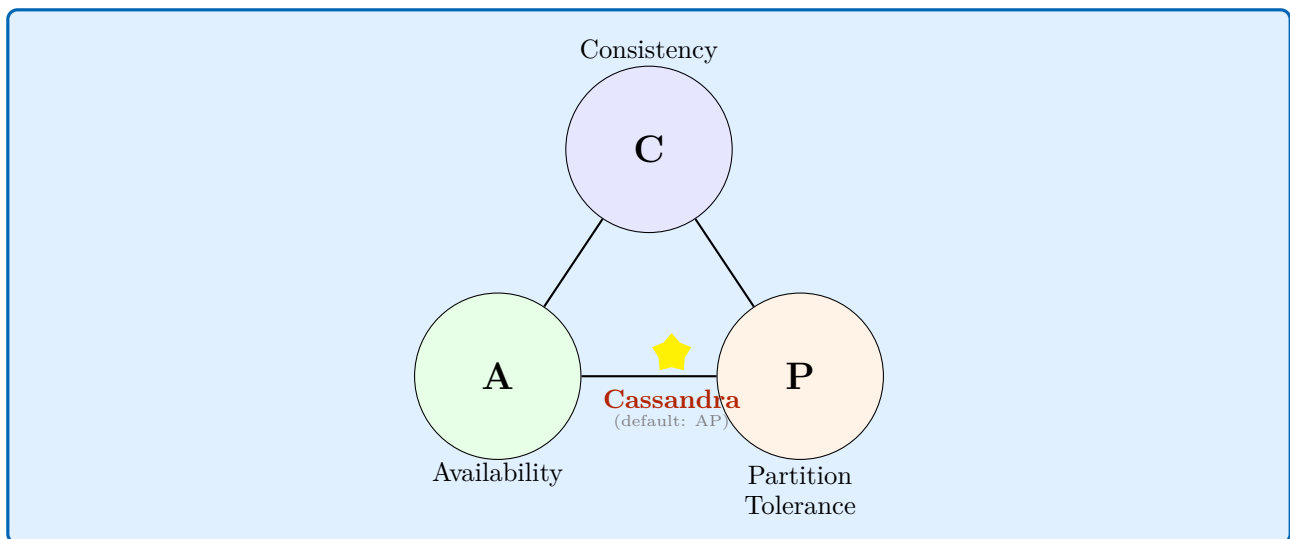
Feature	Explanation
Proven	Used by Apple (75,000+ nodes), Netflix (2,500 nodes), eBay, Instagram, Reddit
Fault Tolerant	Data is automatically replicated. If a node dies, another copy exists elsewhere
Performant	Outperforms many NoSQL alternatives in benchmarks
Decentralised	No single point of failure. No master node. Every node is equal
Scalable	Performance scales <i>linearly</i> — double the nodes, double the throughput
Elastic	Add/remove nodes without taking the system offline
Durable	Data survives even if an entire data centre goes down

Cassandra and the CAP Theorem

Where Does Cassandra Sit on the CAP Triangle?

Cassandra is primarily an **AP** system (Availability + Partition Tolerance). It always tries to give you a response, even if the network is having problems. The trade-off is that the response might be slightly out-of-date (eventual consistency).

However, Cassandra is **tuneable towards CP**. You can increase the consistency level (e.g., set it to QUORUM or ALL) to get stronger consistency at the expense of availability.



Cassandra Architecture

Consistent Hashing (How Data Is Distributed)

The Pizza Delivery Zones Analogy Imagine a pizza chain with 10 delivery areas arranged in a circle, numbered 1–100. Each area covers 10 numbers. When someone orders pizza, their address is converted to a number (hashed). That number tells us which delivery zone (= node) handles the order.

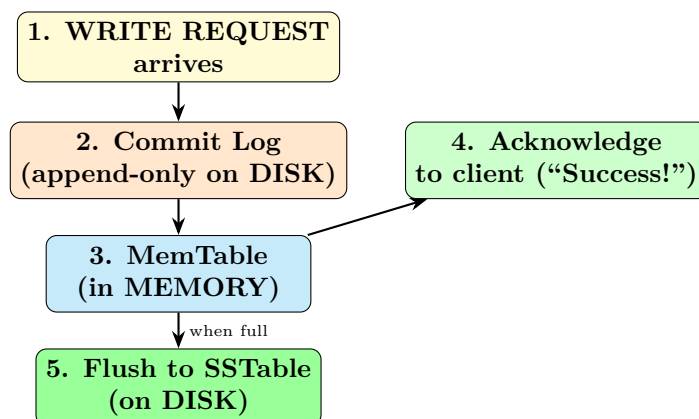
If the hash of your address gives 15, you go to the zone that covers 11–20 (= Node 20). This is exactly how Cassandra distributes data across its ring of nodes.

How Consistent Hashing Works

1. Cassandra arranges all nodes in a virtual **ring**.
2. Each node is assigned a **token** (a position on the ring) and is responsible for a **range of hash values**.
3. When data arrives, the **partition key is hashed** to produce a number.
4. That number maps to a position on the ring, and the data goes to the node responsible for that range.
5. Any node can receive a request (the **coordinator node**), and it forwards the data to the correct node.

Write Path (How Cassandra Writes Data)

When you write data to Cassandra, it goes through these steps:



Write Path Steps (Exam Favourite!)

1. **Commit Log** — Data is immediately written to an append-only log on disk. This is crash-safe — if the machine dies, we can recover from this log. Appending is very fast (no seeking).
2. **MemTable** — Data is also stored in-memory (RAM) for fast access. This is a sorted, in-memory data structure.
3. **Acknowledge** — Cassandra tells the client “write successful!” at this point (very fast!).
4. **SSTable (Sorted Strings Table)** — When the MemTable gets full, it is *flushed* (written) to disk as an **SSTable**. SSTables are *immutable* — once written, they never change.

Compaction (Housekeeping)

What Is Compaction?

Over time, many SSTables accumulate on disk (every MemTable flush creates a new one). Cassandra periodically runs a background process called **compaction** that:

1. **Reads** multiple SSTables
2. **Merges** them together (keeping only the latest version of each row, based on timestamp)
3. **Writes** a new, consolidated SSTable
4. **Deletes** the old SSTables

This keeps disk usage efficient and speeds up reads.

Read Path (How Cassandra Reads Data)

Read Path Steps

1. Check the **MemTable** first (fastest — it is in memory).
2. Check the **Row Key Cache** (cache of frequently accessed rows).
3. Check the **Bloom Filter** — quickly tells us “this SSTable *definitely does not* contain the data” or “it *might* contain the data.” (Avoids reading unnecessary SSTables from disk.)
4. If Bloom Filter says “maybe yes” → check the **Partition Key Cache** → **Partition Summary** → **Partition Index** → read the **SSTable** from disk.
5. Return the result.

Write Path: Consistency Levels

Consistency Level

The **consistency level** controls *how many replica nodes* must confirm a write (or read) before Cassandra tells the client it was successful.

The coordinator node sends the write to *all* replicas, but only waits for a response from the number specified by the consistency level.

Level	What It Means	Trade-off
ANY	At least 1 replica <i>or</i> a hinted handoff (stored note to deliver later)	Most available
ONE	At least 1 actual replica node confirms (hinted handoff NOT enough)	
TWO	At least 2 replica nodes confirm	
THREE	At least 3 replica nodes confirm	
QUORUM	A majority of replicas: $\lfloor N/2 \rfloor + 1$ where N = replication factor	Good balance
ALL	Every single replica node must confirm	Most consistent

QUORUM Calculation **Formula:** $QUORUM = \lfloor \frac{N}{2} \rfloor + 1$

In plain English: Take the replication factor N , divide by 2, round *down*, then add 1.

Worked examples:

- $N = 3$: $QUORUM = \lfloor 3/2 \rfloor + 1 = \lfloor 1.5 \rfloor + 1 = 1 + 1 = 2$
- $N = 5$: $QUORUM = \lfloor 5/2 \rfloor + 1 = \lfloor 2.5 \rfloor + 1 = 2 + 1 = 3$
- $N = 7$: $QUORUM = \lfloor 7/2 \rfloor + 1 = \lfloor 3.5 \rfloor + 1 = 3 + 1 = 4$

The $\lfloor \rfloor$ symbol means “round down to the nearest whole number”. So $\lfloor 1.5 \rfloor = 1$, $\lfloor 2.5 \rfloor = 2$, etc.

Higher Consistency = Lower Availability

- Moving from ANY → ALL: **consistency increases** but **availability decreases**.
- At ALL: if even one replica is down, the write/read **fails**.
- At ANY: writes almost always succeed, but you might read stale data.
- QUORUM is the recommended “sweet spot” for most applications.

Bloom Filters

The Bouncer Analogy Imagine a nightclub with a bouncer who has a very fast but imperfect guest list system. When you arrive, the bouncer checks his system:

- If the system says “**NO**”, you are **definitely not** on the list. Go home. (No false negatives.)
- If the system says “**MAYBE YES**”, you *might* be on the list. Let’s check the full guest list inside to be sure. (Possible false positives.)

This saves time because the bouncer doesn’t need to search the full guest list for every person — only for those the fast system says “maybe.”

What Is a Bloom Filter?

A **Bloom Filter** is a space-efficient, probabilistic data structure used to test whether an element is a member of a set. In Cassandra, each SSTable has an associated Bloom Filter stored in memory.

Key properties:

- **Can give FALSE POSITIVES** (says “maybe present” when it’s not) → must double-check on disk.
- **Can NEVER give FALSE NEGATIVES** (if it says “not present”, it is 100% not there) → skip this SSTable entirely.
- Stored entirely in **memory** → extremely fast to check.

How Bloom Filters Work: Step by Step

Step 1: Setup

Start with a **bit array** (a row of 0s and 1s) of fixed size, initially all zeros. Choose k different hash functions (k is typically 2–3).

Step 2: Inserting an Element

When you insert an element (e.g., the word “aardvark”):

1. Run it through **each of the k hash functions**. Each gives a position number.
2. **Set the bit to 1** at each of those positions.

Inserting “aardvark” (3 hash functions) Bit array size = 16 (positions 1 to 16), all start at 0.

Hash function 1(“aardvark”) = 3

Hash function 2(“aardvark”) = 1

Hash function 3(“aardvark”) = 14

Set positions 1, 3, and 14 to 1:

1	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	

Then Inserting “bat” Hash function 1(“bat”) = 16

Hash function 2(“bat”) = 1 (already 1 — no change)

Hash function 3(“bat”) = 7

Set positions 1, 7, and 16 to 1:

1	0	1	0	0	0	1	0	0	0	0	0	0	1	0	1
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Step 3: Checking if an Element Exists

Run the element through the same k hash functions and check the corresponding bit positions.

Checking “elephant” — Definite NO Hash function 1(“elephant”) = 16 → bit is 1 ✓

Hash function 2(“elephant”) = 2 → bit is 0 ×

Hash function 3(“elephant”) = 7 → bit is 1 ✓

Result: At least one bit is 0, so “elephant” is **definitely NOT** in the set. No need to check the SSTable on disk!

Checking “snake” — False Positive! Hash function 1(“snake”) = 1 → bit is 1 ✓

Hash function 2(“snake”) = 14 → bit is 1 ✓

Hash function 3(“snake”) = 16 → bit is 1 ✓

Result: All bits are 1, so the Bloom Filter says “**MAYBE YES.**” But “snake” was never inserted! Those bits were set by “aardvark” and “bat” coincidentally. This is a **false positive**. We must go check the SSTable on disk to confirm.

Bloom Filter Summary

- All bits = 1 ⇒ Element **might** be present (“Maybe Yes”) → check disk to confirm
- Any bit = 0 ⇒ Element is **definitely not** present (“Definitely No”) → skip this SSTable
- **False Positives: YES** (can happen) **False Negatives: NEVER** (cannot happen)
- In Cassandra: each SSTable has its own Bloom Filter, stored in RAM for speed

Cassandra Query Language (CQL)

CQL looks very similar to SQL, but with important differences. Cassandra uses `COLUMNFAMILY` (or `TABLE`) instead of tables, and has significant restrictions on queries.

Keyspace Operations

Creating and Using a Keyspace

```
-- Create a keyspace with replication settings
CREATE KEYSPACE mykeyspace WITH REPLICATION = {
  'class': 'SimpleStrategy',
  'replication_factor': 3
};

-- Switch to this keyspace (like USE database in MySQL)
USE mykeyspace;

-- View all keyspaces
DESC KEYSPACES;

-- View details of a specific keyspace
DESC mykeyspace;
```

What do the replication settings mean?

- `SimpleStrategy`: Puts replicas on the next nodes clockwise on the ring. Good for single data centre.
- `replication_factor: 3`: Every piece of data is copied to 3 different nodes.

Column Family (Table) Operations

Creating a Column Family

```
-- Create a column family (table)
CREATE COLUMNFAMILY People (
  familyName VARCHAR,
  firstName VARCHAR,
  age INT,
  address VARCHAR,
  PRIMARY KEY (familyName)
);

-- View all tables in the current keyspace
DESC TABLES;

-- View structure of a specific table
DESC People;
```

Note: You can use `TABLE` instead of `COLUMNFAMILY` everywhere — they are interchangeable in CQL.

INSERT, SELECT, UPDATE, DELETE

Full CRUD Operations

```
-- INSERT data (must specify column names)
INSERT INTO People (familyName, firstName, age, address)
VALUES ('Gray', 'Alasdair', 40, 'Edinburgh');

-- INSERT without all columns (age omitted = not stored)
INSERT INTO People (familyName, firstName, address)
VALUES ('Smith', 'John', 'London');

-- SELECT all data
SELECT * FROM People;

-- SELECT with WHERE on primary key
SELECT * FROM People WHERE familyName = 'Gray';

-- SELECT with IN clause
SELECT * FROM People
WHERE familyName IN ('Gray', 'Smith');

-- UPDATE a specific row
UPDATE People
SET phone = '01314523243'
WHERE familyName = 'Gray';

-- DELETE a row
DELETE FROM People WHERE familyName = 'Gray';

-- DROP (delete) the entire column family
DROP COLUMNFAMILY People;
```

INSERT Overwrites in Cassandra! If you INSERT a row with the same primary key as an existing row, Cassandra **overwrites the existing row silently**. In SQL, you would get a “duplicate key” error. In Cassandra, the newer write simply replaces the older one (based on timestamp). This is called an **upsert** (update + insert).

Querying Non-Primary-Key Columns

You CANNOT Filter on Non-Indexed Columns!

```
-- THIS WILL FAIL:
SELECT * FROM People WHERE address = 'Edinburgh';
-- Error: Cannot execute this query as it might involve
-- data filtering...
```

Cassandra does not allow filtering on columns that are not part of the primary key or do not have an index. You have two options:

Option 1: Create a secondary index

```
CREATE INDEX ON People (address);
-- Now the query works:
```

```
SELECT * FROM People WHERE address = 'Edinburgh';
```

Option 2: Use ALLOW FILTERING (bad for performance)

```
SELECT * FROM People  
WHERE address = 'Edinburgh' ALLOW FILTERING;
```

ALTER COLUMNFAMILY (Adding Columns)

Adding a New Column On-the-Fly

```
-- Add a phone column to People  
ALTER COLUMNFAMILY People ADD phone VARCHAR;
```

New columns can be added while the database is running and processing queries — no downtime needed. Existing rows won't have this column until you add data to it (sparse storage!).

Cassandra vs Relational Databases (RDBMS)

Critical Differences

Feature	Cassandra (CQL)	RDBMS (SQL)
JOINS	NOT supported. You must de-normalise data or handle joins in application code.	Fully supported
Foreign Keys	NOT supported. No referential integrity.	Supported
Aggregates	Very limited. Only on primary key columns.	Full support (SUM, AVG, GROUP BY, etc.)
Subqueries	NOT supported.	Supported
Schema	Flexible — rows can have different columns (sparse)	Rigid — all rows have same columns
INSERT behaviour	Upsert (overwrites if same PK)	Error if duplicate PK
Scaling	Horizontal (add more machines)	Vertical (bigger machine)
NULLs	Columns simply don't exist if no value	NULL stored explicitly

Foreign Keys Don't Work in Cassandra

```
-- This FAILS in Cassandra:
CREATE COLUMNFAMILY Employees (
  id VARINT,
  firstName VARCHAR,
  deptId VARINT,
  PRIMARY KEY (id),
  FOREIGN KEY (deptId) REFERENCES Departments (id)  -- ERROR!
);
-- Solution: Just remove the FOREIGN KEY line.
-- Cassandra does not enforce relationships.
```

JOINS Don't Work in Cassandra

```
-- This FAILS in Cassandra:
SELECT * FROM Employees JOIN Departments
ON Employees.deptId = Departments.id;
-- Syntax error!

-- Solution: Query each table separately and join
-- the results in your application code (Python, Java, etc.)
```

Wide Columns: Custom Types and Nested Data

One of Cassandra's powerful features is the ability to nest complex data structures within a single column using **custom types**, **sets**, **lists**, and **maps**.

Custom Types (User-Defined Types)

Creating and Using a Custom Type

```
-- Step 1: Create a custom type called 'address'
CREATE TYPE address (
    street VARCHAR,
    city VARCHAR,
    postalCode VARCHAR,
    phones SET<VARCHAR> -- a set of phone numbers
);

-- Step 2: Check your types
DESC TYPES;
DESC TYPE address;

-- Step 3: Use the type in a column family
-- 'frozen' means the whole value is treated as one blob
CREATE COLUMNFAMILY Users (
    id VARINT,
    name VARCHAR,
    addresses FROZEN<address>,
    PRIMARY KEY (id)
);

-- Step 4: Insert data with the nested type
INSERT INTO Users (id, name, addresses)
VALUES (1, 'Alice Watt', {
    street: '912 Home Street',
    city: 'Edinburgh',
    postalCode: 'EH12 3SK',
    phones: {'01312291823', '0772132912'}
});

-- Step 5: Query all data
SELECT * FROM Users;

-- Step 6: Project nested fields
SELECT addresses.street FROM Users;
```

What Does FROZEN Mean?

The **FROZEN** keyword means Cassandra treats the entire complex type as a **single, immutable blob**. You cannot update individual fields inside a frozen type — you must replace the whole thing. Think of it like a frozen meal: you can't change just one ingredient; you have to replace the entire meal.

Using Maps for Multiple Addresses

Map of Addresses (home, work, etc.)

```
-- Create a table where addresses is a MAP
-- The key is a label (e.g. 'home', 'work')
-- The value is a frozen address type
CREATE COLUMNFAMILY Users2 (
  id VARINT,
  name VARCHAR,
  addresses MAP<TEXT, FROZEN<address>>,
  PRIMARY KEY (id)
);

-- Insert with multiple addresses
INSERT INTO Users2 (id, name, addresses)
VALUES (1, 'Bob', {
  'home': {
    street: '912 Home Street',
    city: 'Edinburgh',
    postalCode: 'EH12 3SK',
    phones: {'01312291823', '0772132912'}
  },
  'work': {
    street: 'HWU drive',
    city: 'Edinburgh',
    postalCode: 'EH14 4AS',
    phones: {'131 4495111'}
  }
});

-- Update just the home address
UPDATE Users2 SET addresses['home'] = {
  street: '10 New Home Street',
  city: 'Edinburgh',
  postalCode: 'EH1 1AB',
  phones: {'0131938273'}
} WHERE id = 1;
```

Collection Types in Cassandra

Type	Syntax	Use Case
SET	SET<VARCHAR>	Unique, unordered values (e.g., phone numbers)
LIST	LIST<VARCHAR>	Ordered values (duplicates allowed)
MAP	MAP<TEXT, INT>	Key-value pairs (e.g., 'home': address)

DataStax Astra & Python Connection

DataStax Astra is a cloud-hosted version of Cassandra. In the lab, you connect to it from Google Colab using the Python Cassandra driver.

Key Steps to Connect

1. Create a Serverless (Non-Vector) database on Astra (e.g., FraudDetectionDB).
2. Create a **keyspace** (e.g., `frauddetectionks`).
3. Generate an **Application Token** (Client ID + Client Secret).
4. Download the **Secure Connect Bundle** (a `.zip` file containing connection certificates).
5. In Python, use the `cassandra-driver` library to connect.

Python Connection Code (Key Parts)

```
from cassandra.cluster import Cluster
from cassandra.auth import PlainTextAuthProvider

# Point to the Secure Connect Bundle
cloud_config = {
    'secure_connect_bundle': 'secure-connect-frauddetectiondb.zip'
}

# Authenticate with Client ID and Secret
auth_provider = PlainTextAuthProvider(
    "your-client-id", "your-client-secret"
)

# Connect
cluster = Cluster(cloud=cloud_config, auth_provider=auth_provider)
session = cluster.connect()

# Switch to your keyspace
session.set_keyspace('frauddetectionks')

# Run a CQL query
rows = session.execute("SELECT * FROM transactions")
for row in rows:
    print(row)

# Filtered query (requires ALLOW FILTERING for non-PK columns)
rows = session.execute(
    "SELECT * FROM transactions "
    "WHERE transaction_amount > 900 ALLOW FILTERING"
)
```

CQL in Python The queries inside `session.execute("...")` are exactly the same CQL queries you would type in the CQL Console. The Python driver just sends them to the database and returns results.

Cassandra vs MongoDB

Criterion	Choose Cassandra	Choose MongoDB
Uptime	100% uptime needed (masterless, no single point of failure)	Some downtime acceptable
Write Performance	Massive write scalability needed	Moderate writes
Data Model	Simple, flat, high-volume data	Rich, nested, document-like data
Secondary Indexes	Weak (single column, equality only)	Strong, flexible indexing
Query Flexibility	Limited (no joins, no subqueries)	Rich query language, aggregation pipeline
Query Language	CQL (SQL-like, easier transition from RDBMS)	MQL (MongoDB Query Language)
Scaling	Linear horizontal scaling	Horizontal via sharding
CAP Position	AP (tuneable towards CP)	CP

CQL Limitations

What CQL Cannot Do

1. **No JOINS or subqueries** — by design, to force denormalisation into partitions that can be queried from a single replica.
2. **Limited aggregation** — `GROUP BY` only works on primary key columns.
3. **A single column value cannot exceed 2 GB** — in practice, keep values in the single-digit MB range.
4. **Maximum 2 billion cells** (rows $\times$ columns) in a single partition.
5. **No foreign keys or referential integrity.**
6. **Secondary indexes are weak** — single columns only, equality comparisons only.

Quick Reference Cheat Sheet

CQL Commands At A Glance

Operation	CQL Syntax
Create keyspace	CREATE KEYSPACE ks WITH REPLICATION = {'class':'SimpleStrategy', 'replication_factor':3};
Use keyspace	USE ks;
Create table	CREATE COLUMNFAMILY T (id VARINT, name VARCHAR, PRIMARY KEY(id));
Insert data	INSERT INTO T (id, name) VALUES (1, 'Alice');
Select all	SELECT * FROM T;
Select with filter	SELECT * FROM T WHERE id = 1;
Update data	UPDATE T SET name='Bob' WHERE id=1;
Delete row	DELETE FROM T WHERE id=1;
Add column	ALTER COLUMNFAMILY T ADD email VARCHAR;
Create index	CREATE INDEX ON T (name);
Create type	CREATE TYPE addr (street VARCHAR, city VARCHAR);
Drop table	DROP COLUMNFAMILY T;
Describe	DESC KEYSPACES; / DESC TABLES; / DESC T;
Batch insert	BEGIN BATCH ... INSERT ... INSERT ... APPLY BATCH;
Consistency	SELECT ... USING CONSISTENCY QUORUM;

Key Concepts Summary

Concept	One-Line Summary
Keyspace	Top-level container (= database). Defines replication strategy.
Column Family	A table. Contains rows with potentially different columns per row.
Partition Key	Part of PK that determines which node stores the data.
Clustering Column	Part of PK that determines sort order within a partition.
Commit Log	Append-only log on disk for crash recovery.
MemTable	In-memory sorted data structure for recent writes.
SSTable	Immutable on-disk file created when MemTable is flushed.
Compaction	Background merge of multiple SSTables into one.
Bloom Filter	Probabilistic test: “definitely not here” or “maybe here.” No false negatives.
Consistent Hashing	Ring-based distribution of data across nodes using hash of partition key.
QUORUM	$\lfloor N/2 \rfloor + 1$ replicas must respond. Balances consistency and availability.
Frozen	Complex type treated as single immutable blob. Must replace entirely.
Hinted Handoff	Coordinator stores a hint for a down node; delivers when it comes back.

Practice Questions

Q1: Data Model What is the difference between a Keyspace and a Column Family in Cassandra?

Answer: A Keyspace is the top-level container (equivalent to a database in RDBMS) that defines replication settings. A Column Family (or Table) is a container for rows within a keyspace. A keyspace can contain many column families.

Q2: Bloom Filters If a Bloom Filter says a key is present, does that guarantee the key is actually in the SSTable?

Answer: No. Bloom Filters can give false positives — they might say “maybe present” when the key is not actually there. However, if the Bloom Filter says “not present,” that is guaranteed to be correct (no false negatives).

Q3: QUORUM Calculation A Cassandra cluster has a replication factor of 5. What is the QUORUM?

Answer: $\text{QUORUM} = \lfloor 5/2 \rfloor + 1 = \lfloor 2.5 \rfloor + 1 = 2 + 1 = 3$. At least 3 out of 5 replica nodes must acknowledge the write/read.

Q4: Cassandra vs RDBMS Why do JOINS not work in Cassandra? What is the alternative?

Answer: Cassandra is designed for distributed data across many nodes. JOINS would require gathering data from multiple partitions across the cluster, which would be extremely slow and defeat the purpose of distributed storage. The alternative is to denormalise your data (store related data together in one table) or perform joins in your application code after querying each table separately.

Q5: Write Path Describe the four steps of Cassandra’s write path.

Answer: (1) Write to the **Commit Log** on disk (append-only, crash-safe). (2) Write to the **MemTable** in memory. (3) **Acknowledge** success to the client. (4) When MemTable is full, **flush** it to an immutable **SSTable** on disk.

Q6: CAP Theorem Where does Cassandra sit on the CAP triangle and why?

Answer: Cassandra is primarily an **AP** system (Availability + Partition Tolerance). It prioritises being always available and handling network partitions, at the cost of eventual consistency. However, it is tuneable towards CP by increasing the consistency level (e.g., QUORUM or ALL).

Q7: INSERT Behaviour What happens if you INSERT a row with the same primary key as an existing row?

Answer: Cassandra performs an **upsert** — it silently overwrites the existing row with the new data. In a traditional RDBMS, this would cause a duplicate key error.

Q8: Frozen Types What does the FROZEN keyword mean in Cassandra?

Answer: FROZEN means the complex type (list, set, map, or user-defined type) is treated as a single immutable blob. You cannot update individual fields inside it — you must replace the entire value. Cassandra serialises the whole frozen value together.